Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Современные операционные системы и базы данных»

на тему:
«Связи таблиц и JOIN в PostgreSQL»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверила
Преподаватель:
доцент, к.н., Галлини
Надежда Игоревна

Оценка:_____
Дата: 29.10.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА
Тема: Связи таблиц и JOIN в PostgreSQL

Цель работы: создать модель связи «многие ко многим» для проектов портфолио, получить корректные результаты INNER JOIN и LEFT JOIN, освоить группировку и поиск отсутствующих связей.

Учебная среда: SQLite 3.50 (файл app/lab09.db) как локальный эквивалент схемы lab09 без установки PostgreSQL/pgAdmin; выполнение выполнения — Python sqlite3 и Git Bash. Файл answers.sql сохранён в синтаксисе PostgreSQL (lab09.*). Пароли и .env не использовались.

1. Подготовка и проверка подключения

Создан файл базы lab09.db. Аналог проверки current_database()/current_user и наличия схемы lab09:

```
sqlite3 lab09.db — 1. Проверка подключения
1. Проверка подключения

sqlite> -- аналог current_database / current_user
current_database | current_user
-----------------+-------------
lab09.db         | sqlite
(1 row)

sqlite> -- схема lab09 = файл lab09.db
schema_name
-----------
lab09
(1 row)
```

Рисунок 1 – Проверка подключения и схемы lab09

2. Создание структуры и данных

Выполнен скрипт 01_schema_data.sql: таблицы projects, technologies, project_technologies (связь многие-ко-многим). Ожидаемое количество строк — по 4 в каждой таблице.

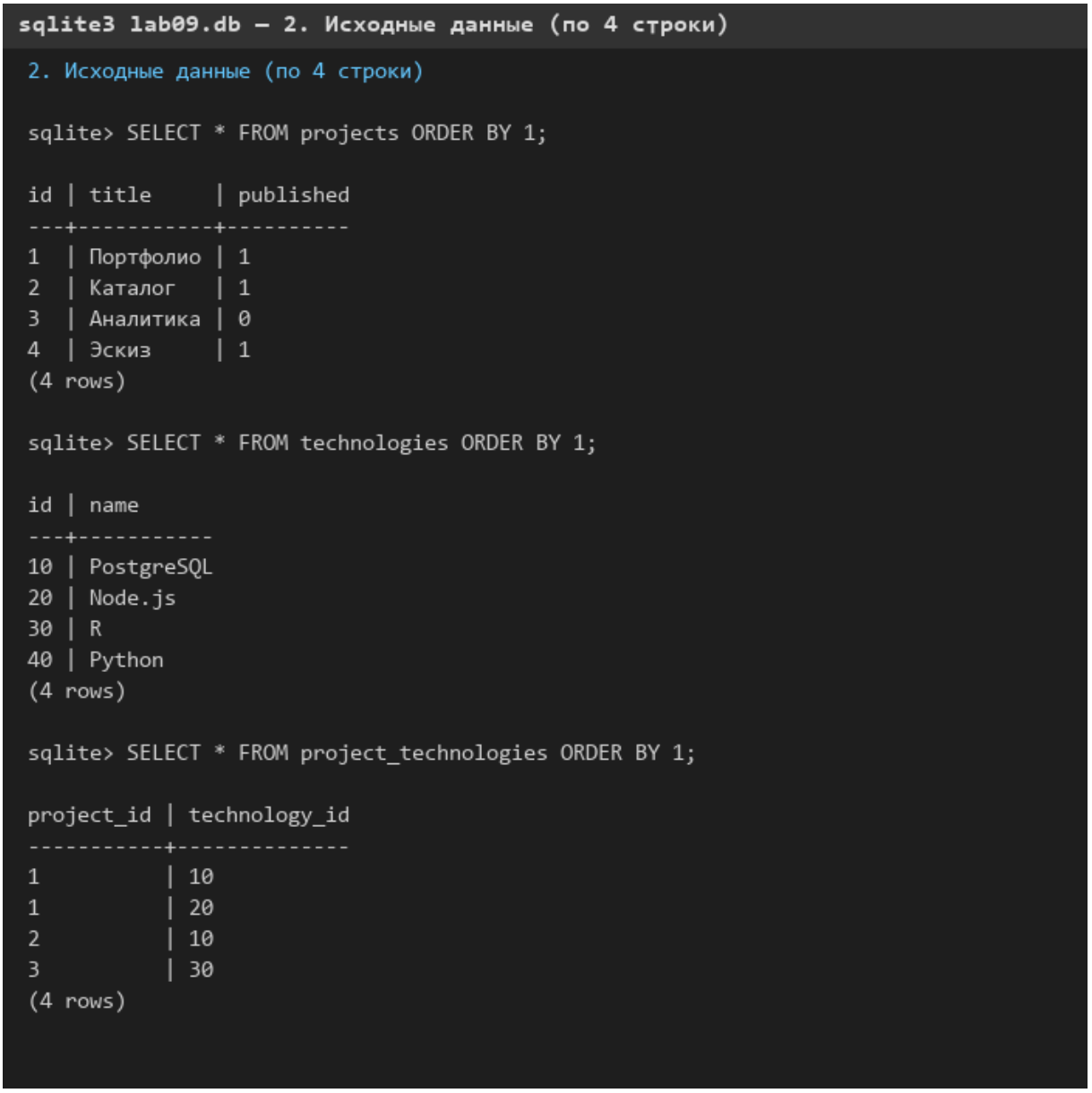


Рисунок 2 – Исходные данные (SELECT … ORDER BY)

Схема связей. Первичные ключи: projects.id, technologies.id, составной PK (project_id, technology_id). Внешние ключи: project_id → projects(id), technology_id → technologies(id). Кратность M:N через таблицу связей; отсутствие связи допустимо (пример — проект «Эскиз»).

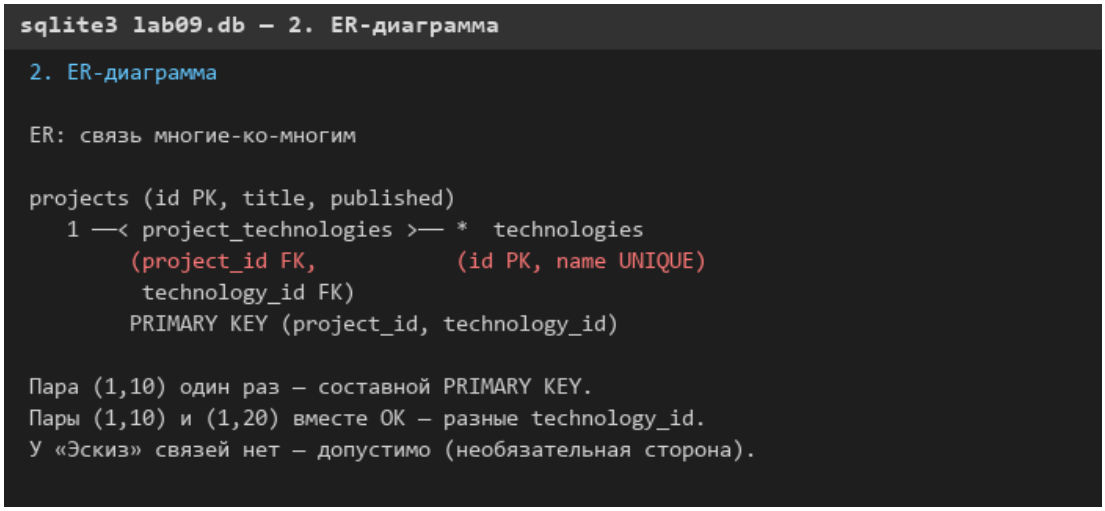


Рисунок 3 – ER-диаграмма

Почему пара (1, 10) допустима только один раз: составной PRIMARY KEY запрещает дубликат одной и той же связи. Пары (1, 10) и (1, 20) могут существовать вместе, потому что technology_id различается — у одного проекта несколько технологий.

3. Соединение таблиц

Прогноз до запуска: Q1 (INNER JOIN) — 4 строки (все существующие связи); Q2 (LEFT JOIN) — 5 строк (4 связи + «Эскиз» с NULL).

```
sqlite3 lab09.db — Q1 INNER JOIN (прогноз 4 → факт 4)
Q1 INNER JOIN (прогноз 4 → факт 4)

sqlite> INNER JOIN ...

title     | name
----------+-----------
Портфолио | PostgreSQL
Портфолио | Node.js
Каталог   | PostgreSQL
Аналитика | R
(4 rows)
```

Рисунок 4 – Q1 INNER JOIN, 4 строки

```
sqlite3 lab09.db — Q2 LEFT JOIN (прогноз 5 → факт 5; Эскиз→NULL)
Q2 LEFT JOIN (прогноз 5 → факт 5; Эскиз→NULL)

sqlite> LEFT JOIN ...

title     | name
----------+-----------
Портфолио | PostgreSQL
Портфолио | Node.js
Каталог   | PostgreSQL
Аналитика | R
Эскиз     | NULL
(5 rows)
```

Рисунок 5 – Q2 LEFT JOIN, 5 строк; у «Эскиз» name = NULL

Строка «Эскиз» появляется только при LEFT JOIN: у проекта нет записей в project_technologies, поэтому поля правой таблицы заполняются NULL. INNER JOIN отбрасывает проекты без совпадений.

Сравнение фильтра WHERE и условия в ON. WHERE t.name = 'PostgreSQL' отфильтровывает результат после соединения и оставляет 2 строки. То же условие в ON второго LEFT JOIN не отбрасывает исходные строки левой стороны: остаётся 5 строк, у «Портфолио» связь с Node.js превращается в NULL. Перенос условия в ON не даёт автоматически одну строку на проект — кратность определяется числом подходящих связей в таблице-связке.

```
sqlite3 lab09.db — Фильтр WHERE t.name='PostgreSQL' → 2 строки
Фильтр WHERE t.name='PostgreSQL' → 2 строки

sqlite> ... WHERE t.name='PostgreSQL'

title     | name
----------+-----------
Портфолио | PostgreSQL
Каталог   | PostgreSQL
(2 rows)
```

Рисунок 6 – Условие в WHERE → 2 строки

```
sqlite3 lab09.db — Условие в ON второго JOIN → 5 строк
Условие в ON второго JOIN → 5 строк

sqlite> ... ON ... AND t.name='PostgreSQL'

title     | name
----------+-----------
Портфолио | NULL
Портфолио | PostgreSQL
Каталог   | PostgreSQL
Аналитика | NULL
Эскиз     | NULL
(5 rows)
```

Рисунок 7 – Условие в ON → 5 строк

4. Группировка и отсутствие связей

Q3: счётчики технологий по проектам — 2, 1, 1, 0. Q4 находит «Эскиз». Q5 (проекты со связью PostgreSQL) возвращает «Портфолио» и «Каталог».

```
sqlite3 lab09.db — Q3 счётчики 2,1,1,0
Q3 счётчики 2,1,1,0

sqlite> COUNT(pt.technology_id)

id | title     | n
---+-----------+--
1  | Портфолио | 2
2  | Каталог   | 1
3  | Аналитика | 1
4  | Эскиз     | 0
(4 rows)
```

Рисунок 8 – Q3: COUNT = 2, 1, 1, 0

```
sqlite3 lab09.db — COUNT(*) vs COUNT(key): у Эскиза 1 и 0
COUNT(*) vs COUNT(key): у Эскиза 1 и 0

sqlite> COUNT(*) , COUNT(pt.technology_id)

id | title     | count_star | count_key
---+-----------+------------+----------
1  | Портфолио | 2          | 2
2  | Каталог   | 1          | 1
3  | Аналитика | 1          | 1
4  | Эскиз     | 1          | 0
(4 rows)
```

Рисунок 9 – COUNT(*) и COUNT(ключа) после LEFT JOIN

Для «Эскиз» COUNT(*) даёт 1: LEFT JOIN сохраняет строку проекта с NULL справа, и эта одна строка учитывается. COUNT(pt.technology_id) игнорирует NULL и даёт 0 — реальное число связей. Поэтому для подсчёта связей после LEFT JOIN нужен COUNT по правому ключу, а не COUNT(*).

```
sqlite3 lab09.db — Q4 проекты без технологий → Эскиз
Q4 проекты без технологий → Эскиз

sqlite> LEFT JOIN ... IS NULL

id | title
---+------
4  | Эскиз
(1 row)
```

Рисунок 10 – Q4: проект без технологий

```
sqlite3 lab09.db — Q5 проекты с PostgreSQL → Портфолио, Каталог
Q5 проекты с PostgreSQL → Портфолио, Каталог

sqlite> INNER JOIN ... WHERE PostgreSQL

id | title     | name
---+-----------+-----------
1  | Портфолио | PostgreSQL
2  | Каталог   | PostgreSQL
(2 rows)
```

Рисунок 11 – Q5: Портфолио и Каталог (PostgreSQL)

```
sqlite3 lab09.db — Листинг 4: HAVING >= 2 → Портфолио
Листинг 4: HAVING >= 2 → Портфолио

sqlite> HAVING COUNT(...)>=2

id | title     | n
---+-----------+--
1  | Портфолио | 2
(1 row)
```

Рисунок 12 – Листинг 4: HAVING COUNT ≥ 2

5. Самостоятельные задания

Решения сохранены в app/answers.sql (синтаксис PostgreSQL). Результаты выполнения:

Задание 1. Все технологии и число проектов (Python с нулём):

```
sqlite3 lab09.db — Задание 1: технологии и число проектов
Задание 1: технологии и число проектов

sqlite> LEFT JOIN + COUNT

id | name       | projects_count
---+------------+---------------
10 | PostgreSQL | 2
20 | Node.js    | 1
30 | R          | 1
40 | Python     | 0
(4 rows)
```

Рисунок 13 – Задание 1

Задание 2. Проекты с двумя и более технологиями:

```
sqlite3 lab09.db — Задание 2: проекты с >=2 технологиями
Задание 2: проекты с >=2 технологиями

sqlite> HAVING >= 2

id | title     | n
---+-----------+--
1  | Портфолио | 2
(1 row)
```

Рисунок 14 – Задание 2

Задание 3. Опубликованные проекты без технологий:

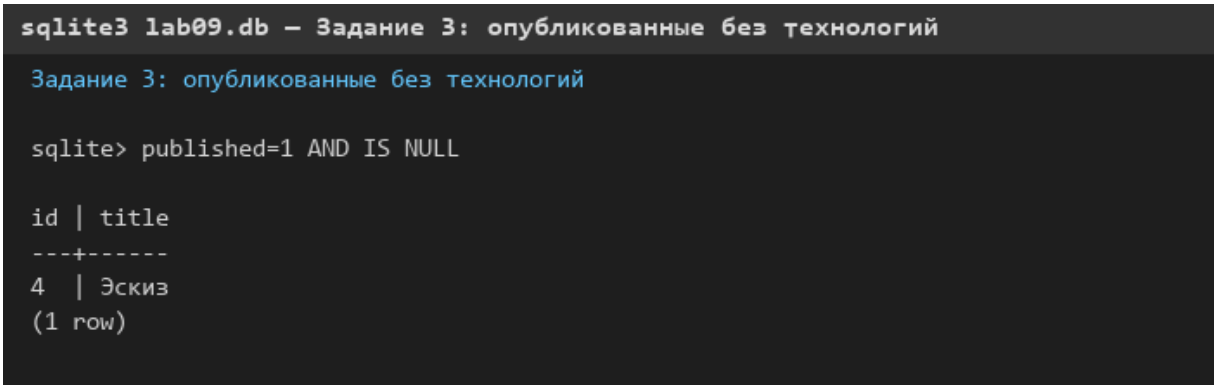


Рисунок 15 – Задание 3

Задание 4. Неиспользуемые технологии через NOT EXISTS:

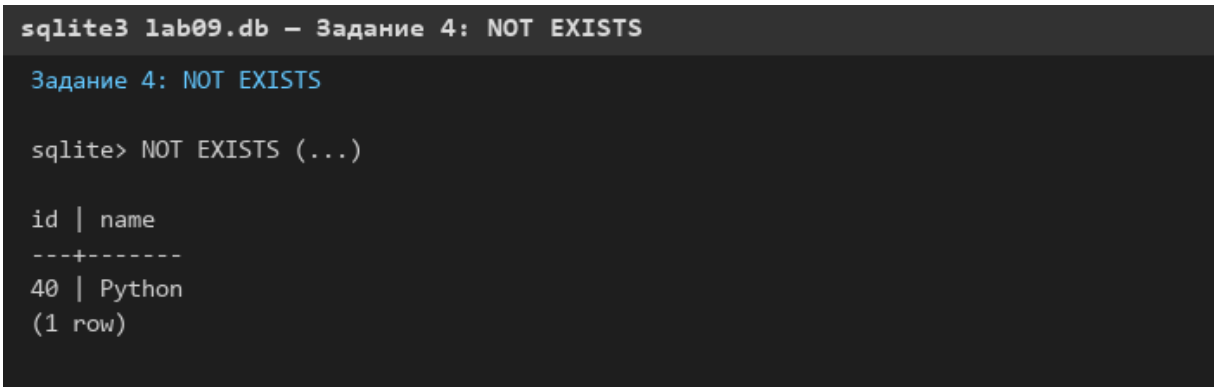


Рисунок 16 – Задание 4

Задание 5. В транзакции добавлен проект id=5 «Учебный прототип» и две связи (PostgreSQL, Python). После проверки выполнен ROLLBACK — количества строк вернулись к 4/4/4.

```
sqlite3 lab09.db — Задание 5: транзакция и ROLLBACK
Задание 5: транзакция и ROLLBACK

sqlite> INSERT project 5 + 2 links; SELECT

id | title            | name
---+------------------+----------
5  | Учебный прототип | PostgreSQL
5  | Учебный прототип | Python
(2 rows)

До ROLLBACK:
tbl                | n
-------------------+--
projects           | 5
technologies       | 4
project_technologies | 6
(3 rows)

После ROLLBACK (ожидание 4/4/4):
tbl                | n
-------------------+--
projects           | 4
technologies       | 4
project_technologies | 4
(3 rows)
```

Рисунок 17 – Задание 5: транзакция и ROLLBACK

Задание 6. На строке «Эскиз» после LEFT JOIN: COUNT(*) = 1 (есть строка результата), COUNT(pt.technology_id) = 0 (нет ненулевых значений ключа).

```
sqlite3 lab09.db — Задание 6: Эскиз — COUNT(*)=1, COUNT(fk)=0
Задание 6: Эскиз — COUNT(*)=1, COUNT(fk)=0

sqlite> WHERE id=4

id | title | count_star | count_fk
---+-------+------------+---------
4  | Эскиз | 1          | 0
(1 row)
```

Рисунок 18 – Задание 6

Задание 7. Каждый проект ровно один раз; имя технологии PostgreSQL только при наличии связи, иначе NULL. Использована заранее отфильтрованная правая таблица (подзапрос).

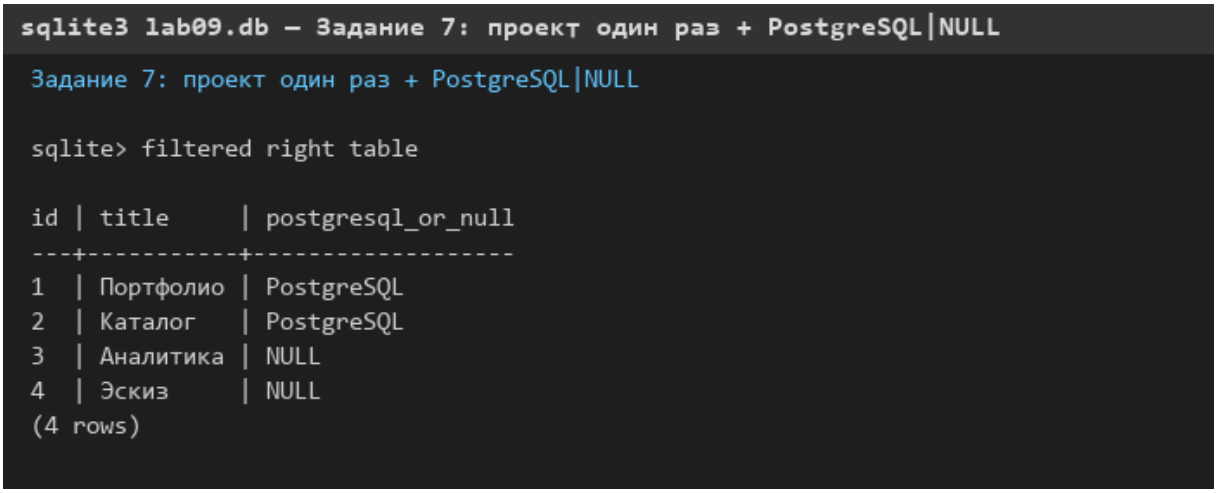


Рисунок 19 – Задание 7

Фрагмент answers.sql (самостоятельные запросы):

```
-- 1
SELECT t.id, t.name, COUNT(pt.project_id) AS projects_count
FROM lab09.technologies t
LEFT JOIN lab09.project_technologies pt ON pt.technology_id = t.id
GROUP BY t.id, t.name ORDER BY t.id;

-- 2
SELECT p.id, p.title, COUNT(pt.technology_id) AS n
FROM lab09.projects p
LEFT JOIN lab09.project_technologies pt ON pt.project_id = p.id
GROUP BY p.id, p.title
HAVING COUNT(pt.technology_id) >= 2 ORDER BY p.id;

-- 3
SELECT p.id, p.title
FROM lab09.projects p
LEFT JOIN lab09.project_technologies pt ON pt.project_id = p.id
WHERE p.published = true AND pt.project_id IS NULL ORDER BY p.id;

-- 4
SELECT t.id, t.name FROM lab09.technologies t
WHERE NOT EXISTS (
  SELECT 1 FROM lab09.project_technologies pt WHERE pt.technology_id = t.id
) ORDER BY t.id;

-- 7
SELECT p.id, p.title, t.name AS postgresql_or_null
FROM lab09.projects p
LEFT JOIN (
  SELECT pt.project_id, t.name
  FROM lab09.project_technologies pt
  JOIN lab09.technologies t ON t.id = pt.technology_id
  WHERE t.name = 'PostgreSQL'
) t ON t.project_id = p.id
ORDER BY p.id;
```

6. Проверка ограничений без изменения данных

Повтор INSERT (1, 10) вызывает нарушение уникальности составного первичного ключа (в SQLite: UNIQUE constraint failed). INSERT с project_id = 999 нарушает внешний ключ (FOREIGN KEY constraint failed), так как родителя нет в projects. В обоих случаях выполнен ROLLBACK; исходные данные не изменены.

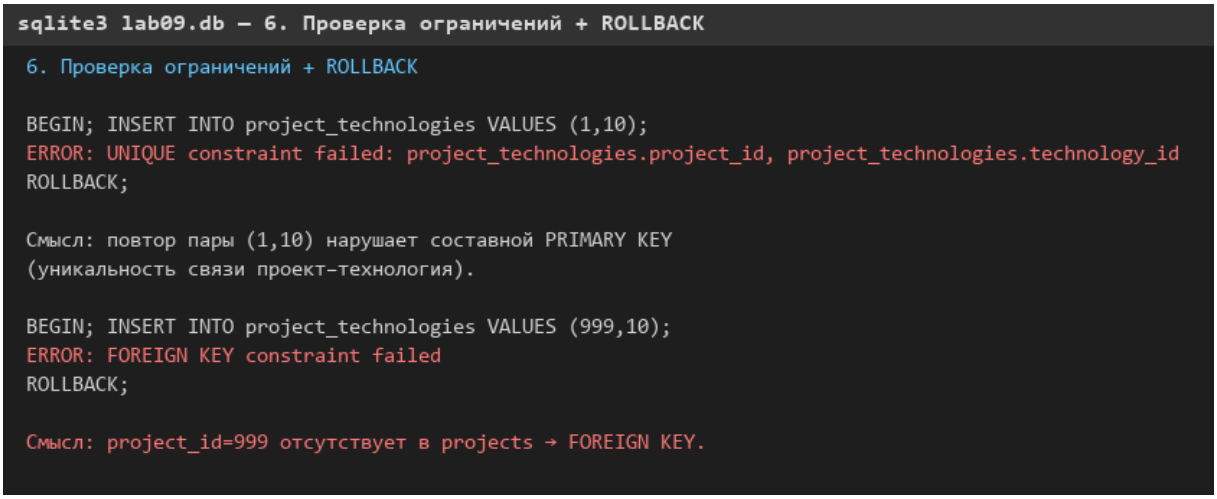


Рисунок 20 – Нарушения PK/FK и откат

7. Выводы

Построена модель M:N через таблицу связей с PK и FK. INNER JOIN показывает только существующие связи; LEFT JOIN сохраняет проекты без технологий (NULL). Фильтр в WHERE сужает итоговый набор, а условие в ON меняет условие сопоставления, не отбрасывая левую сторону LEFT JOIN. Для подсчёта связей после LEFT JOIN следует использовать COUNT по правому ключу. Ограничения уникальности и внешних ключей проверены в отдельных транзакциях с ROLLBACK.

Ответы на контрольные вопросы

1) «Эскиз» исчезает после INNER JOIN, потому что нет совпадений в project_technologies, а INNER JOIN оставляет только совпавшие строки.

2) COUNT(*) даёт 1 для «Эскиз» после LEFT JOIN: строка проекта остаётся в результате с NULL справа и учитывается как одна строка группы.

3) WHERE фильтрует уже соединённый результат; ON задаёт условие сопоставления строк при соединении.

4) Внешний ключ требует существования родителя, но не ограничивает число дочерних строк с одним и тем же project_id — у проекта может быть несколько технологий.

5) EXISTS/NOT EXISTS удобны для проверки наличия/отсутствия связанных строк без раздувания результата декартовым эффектом JOIN.